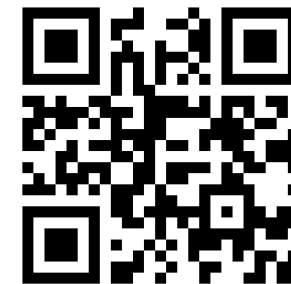




KCD BRAZIL, SÃO PAULO 2026

KCD nas redes!



Do API Server ao ETCD:

Anatomia, Gargalos e o Futuro dos Admission Controllers

Roberto Scudeller

Senior Infrastructure Engineer · Independente



KCD BRAZIL · SÃO PAULO 2026

Do API Server ao ETCD

Kubernetes Admission Controllers

Silver



Gold



Sponsors

Diamond



RealCloud



OLÁ, EU SOU



Roberto Scudeller

Senior Infrastructure Engineer · Independente

- "DevOps" desde 2012, ou TI desde 2006 (Operador de Data Center conta né?)
- Trabalhei como Linux SysAdmin até Cloud Security Engineer
- 5 anos morando e trabalhando com TI na Alemanha (visto de trabalho de TI)
- Últimos 8 anos trabalhando para empresas internacionais

Nessa talk de hoje



01

Da Autenticação ao ETCD

Do início da requisição ao ETCD, passando pelos controles de admissão

02

Admission Policies

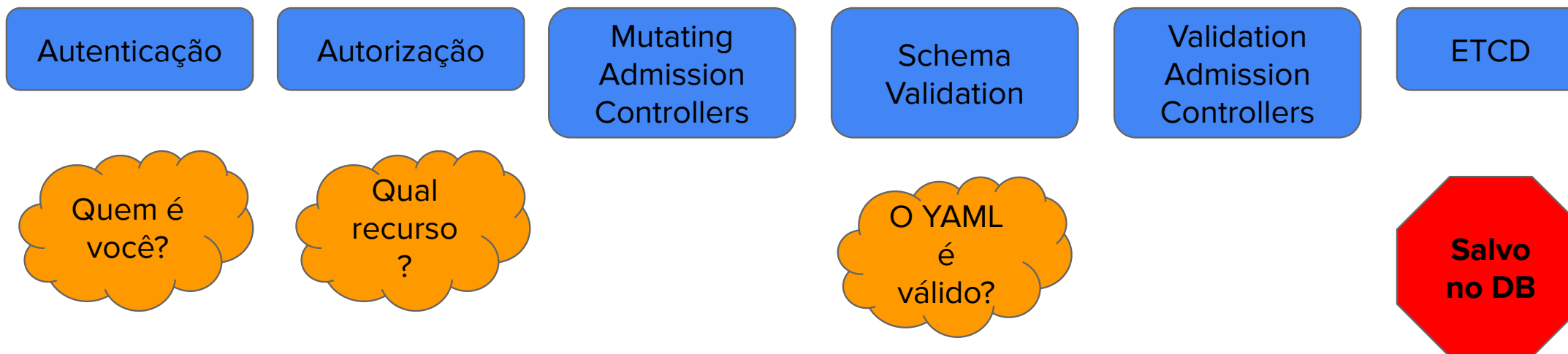
Configurações, linguagem e dicas.

03

Aprendizados e próximos passos

O que levar desta conversa para o dia a dia.

Da Autenticação ao ETCD



Admission Controllers as flags



- "--enable-admission-plugins strings": grande lista de plugins, oposto de "--disable-admission-plugins strings"
- "--admission-control-config-file string": caso queira configurar direto no API Server
- Casos de Uso?

EM RESUMO

Opções para controlar admission plugins direto no API Server.

Mutating Admission Controllers

- Default Ingress Class
- Default Storage Class
- Priority
- Service Account
- e muitas outras como Image Pull Policy, DNS Policy, Termination Grace Period...

EM RESUMO

Mutating admission controllers que estão presentes mesmo que você não configure nenhum.



Validating Admission Controllers

- Namespace Exist
- Node Restriction
- Resource Quota
- Priority
- E muitos outros

EM RESUMO

Validating admission controllers sempre acontecem após mutating e validação de schema. Bom lugar para regras invioláveis.



Kubernetes Recursos

- Validating Admission Policy
- Validating Admission Policy Binding
- Mutating Admission Policy
- Mutating Admission Policy Binding
- Usam CEL (Common Expression Language)
- Mutating Webhook Configuration
- Validating Webhook Configuration.

EM RESUMO

Funcionam semelhante a Role and RoleBinding



Common Expression Language

- É deliberadamente limitada: Sem loops, sem recursão ilimitada.
- Exemplos:
 - `"has(object.metadata.labels) && 'app.kubernetes.io/name' in object.metadata.labels"`
 - `"object.kind != 'Pod' || object.spec.containers.all(container, has(container.securityContext) && has(container.securityContext.capabilities) && has(container.securityContext.capabilities.drop) && container.securityContext.capabilities.drop.exists(c, c == 'ALL'))"`

EM RESUMO

Criada para ser segura de executar no API Server



Exemplos



mutating-pol-gotham.yaml

```
1  apiVersion: admissionregistration.k8s.io/v1
2  kind: MutatingAdmissionPolicy
3  metadata:
4    name: gotham-required-label
5  spec:
6    failurePolicy: Fail
7    reinvocationPolicy: Never
8    matchConstraints:
9      resourceRules:
10       - apiGroups: ["*"]
11         apiVersions: ["*"]
12         operations: ["CREATE"]
13         resources: ["*"]
14      matchConditions:
15       - name: "missing-required-label"
16         expression: "!has(object.metadata.labels) || !('super.hero/team' in
17           object.metadata.labels)"
```

O QUE ESTE EXEMPLO FAZ

Cria uma regra de alteração (de labels).

DESTAQUE E RESULTADO

Seleciona os recursos que serão aplicados estas regras

Exemplos



O QUE ESTE EXEMPLO FAZ

Adiciona o label

DESTAQUE E RESULTADO

Assim que associarmos um "binding" a esta policy, todos os recursos filtrados terão um campo de label adicionado.

mutating-pol-gotham.yaml

```
1
2  apiVersion: admissionregistration.k8s.io/v1
3  kind: MutatingAdmissionPolicy
4  metadata:
5    name: gotham-required-label
6  spec:
7    ...
8  mutations:
9    - patchType: "ApplyConfiguration"
10     applyConfiguration:
11       expression: |
12         Object{
13           metadata: Object.metadata{
14             labels: {'super.hero/team': 'batman'}
15           }
16         }
```

Exemplos



● ● ● binding-mutating-gotham.yaml

```
1
2 apiVersion: admissionregistration.k8s.io/v1
3 kind: MutatingAdmissionPolicyBinding
4 metadata:
5   name: bind-add-gotham-required-namespace-label
6 spec:
7   policyName: gotham-required-label
8   matchResources:
9     namespaceSelector:
10       matchLabels:
11         city: gotham
```

O QUE ESTE EXEMPLO FAZ

Aplica um "binding" em qualquer namespace que tenha este label.

DESTAQUE E RESULTADO

Adição de labels customizados em todos os recursos criados.

Webhooks e Policies



- Mutating Admission Webhook e Validating Admission Webhook estendem a sua alteração e/ou validação. Requerem TLS!
- Quando usar?
- Dica ao implantar: "failurePolicy: Ignore" para testes de acesso entre Kube API Server e seu novo Webhook no Cluster
- Verifique as métricas sempre:
"apiserver_admission_webhook_admission_duration_seconds"
(latência do seu webhook)
"apiserver_validating_admission_policy_check_duration_seconds" (pode user mutating, ou alterar o final "policy_check_total")

EM RESUMO

Amplie sua cobertura de validações caso necessário com webhooks.

ACESSE AGORA

Referências e exemplos



Repositório git com exemplos de Mutating e Validating Admission policies e as referências desta apresentação.

<https://github.com/betorvs/kcd-shared-contents/tree/main/api-Server-Etcd-admission-controllers-26-09-26>



TAKEAWAYS

O que aprendemos



Common Expression language não é um bicho de 7 cabeças



Quando usar um webhook e quando não usar



Acesse meu repositório e use as referências para criar suas próprias políticas.



Perguntas?

Obrigado

Roberto Scudeller · <https://sessionize.com/roberto-scudeller/>

<https://www.linkedin.com/in/robertoscudeller/>



Comunidade KCD, parte da CNCF · community.cncf.io/kcd-brasil

Pesquisa de satisfação da sessão!



Sponsors

KCD BRAZIL / SÃO PAULO 2026

Diamond



Red Hat



BROADCOM®

null/platform



RealCloud



Kong

Gold



appoena



opsteam



Silver



Grafana

